

Python Radio 45: Amazing Rodent Control!

Using an Accelerometer to Build a Bluetooth Mouse

Simon Quellen Field

Simon Quellen Field

Follow

9 min read

.

Jun 25, 2025

Listen

Share

More

Press enter or click to view image in full size

A mouse is controlling a microcomputer by balancing on it.

MidJourney

I have a young neighbor who is learning Python in grammar school.

He has a BBC Micro:bit and likes computer games.

Plan A

Why not show him how to use a BBC Micro:bit as a computer mouse and write a maze game to use it as a controller? You would tilt the microcomputer to move the player, like the old marble labyrinth games.

I spent a day trying to get the Micro:bit to function as a Bluetooth mouse, but

it turned out to be beyond its capabilities. The accelerometer worked fine, but there is no Bluetooth support for becoming an HID mouse.

Plan B

I rethought the project and ordered a three-pack of MPU6050 modules for \$10 on Amazon. My Seeed Studios XIAO ESP32-C3 became the new microcontroller. It's a lot smaller than the Micro:bit, but lacks the built-in array of sensors and indicators the Micro:bit has.

But now it has a 3-axis accelerometer and 3-axis gyroscope, thanks to the MPU6050.

Press enter or click to view image in full size

The new mouse.

The new mouse.

The Maze Game (Theseus and the Minotaur)

While waiting for the part to arrive, I wrote the maze game. Then, since it hadn't arrived yet, I added a Minotaur to chase the player. If the Minotaur caught up to Theseus before Theseus could figure out the maze and escape, Theseus would lose.

The Minotaur secretly marks any dead ends he finds so that he won't go there anymore. But that only works if the maze has no loops. My maze has loops if you select to make it easier to solve. I had to add some extra intelligence to the Minotaur to give him a chance of catching Theseus.

Because I had all this extra time, the code grew larger than I originally planned, and my young friend will probably wait a few years before trying to Grok it in fullness. Oh well.

Here is the game (you can play it with a regular mouse while waiting for your MPU6050 and ESP32-C3 to arrive):

```
Import tkinter as tk
```

```
From tkinter import messagebox
```

```
Import random
```

Import argparse

Class Maze:

```
Def __init__(self, width, height, difficulty=0.1):
```

```
    Self.width = width; self.height = height
```

```
    Self.difficulty = max(0.0, min(difficulty, 1.0))
```

```
    Self.grid = self._initialize_grid()
```

```
    Self._generate_maze()
```

```
    Self._add_loops()
```

```
Def _initialize_grid(self):
```

```
    Return [[{'N': True, 'S': True, 'E': True, 'W': True, 'visited': False} for _ in  
range(self.width)] for _ in range(self.height)]
```

```
Def _generate_maze(self):
```

```
    Stack = []; x, y = random.randint(0, self.width-1), random.randint(0,  
self.height-1)
```

```
    Self.grid[y][x]['visited'] = True; stack.append((x, y))
```

```
    While stack:
```

```
        X, y = stack[-1]; neighbors = []
```

```
        If y > 0 and not self.grid[y-1][x]['visited']: neighbors.append('N')
```

```
        If y < self.height - 1 and not self.grid[y+1][x]['visited']:  
neighbors.append('S')
```

```
        If x < self.width - 1 and not self.grid[y][x+1]['visited']:  
neighbors.append('E')
```

```
        If x > 0 and not self.grid[y][x-1]['visited']: neighbors.append('W')
```

```
        If neighbors:
```

```
            Direction = random.choice(neighbors)
```

```
            If direction == 'N': self.grid[y][x]['N'] = self.grid[y-1][x]['S'] = False;  
ny, nx = y - 1, x
```

```
            Elif direction == 'S': self.grid[y][x]['S'] = self.grid[y+1][x]['N'] = False;  
ny, nx = y + 1, x
```

```
            Elif direction == 'E': self.grid[y][x]['E'] = self.grid[y][x+1]['W'] = False;
```

```

ny, nx = y, x + 1
    Elif direction == 'W': self.grid[y][x]['W'] = self.grid[y][x-1]['E'] = False;
ny, nx = y, x - 1
    Self.grid[ny][nx]['visited'] = True; stack.append((nx, ny))
    Else: stack.pop()

Def _add_loops(self):
    Num_walls = int(self.difficulty * self.width * self.height)
    For _ in range(num_walls):
        X, y = random.randint(0, self.width-2), random.randint(0, self.height-2)
        If random.choice([True, False]):
            If self.grid[y][x]['S']: self.grid[y][x]['S'] = self.grid[y+1][x]['N'] = False
        Else:
            If self.grid[y][x]['E']: self.grid[y][x]['E'] = self.grid[y][x+1]['W'] = False

Class MazeGame:
    Def __init__(self, maze, cell_size=20, minotaur_speed_ms=150):
        Self.maze = maze
        Self.cell_size = cell_size
        Self.minotaur_speed = minotaur_speed_ms
        Self.player_speed = 200 # How often to check for player movement, in
ms
        Self.width_px = self.maze.width * self.cell_size
        Self.height_px = self.maze.height * self.cell_size

        Self.window = tk.Tk()
        Self.window.title("Theseus Game – RACE THE MINOTAUR!")
        Self.canvas = tk.Canvas(self.window, width=self.width_px,
height=self.height_px, bg='white')
        Self.canvas.pack()

```

```
Self.player_pos = (self.maze.width // 2, self.maze.height // 2)
```

```
Self.player_icon = None
```

```
Self.minotaur_pos = (0, 0)
```

```
Self.minotaur_path = [self.minotaur_pos]
```

```
Self.minotaur_icon = None
```

```
Self.dead_ends = set()
```

```
Self.mouse_pos = (0, 0) # Track the last mouse position
```

```
Self.game_over = False
```

```
Self.game_active = False
```

```
Self.canvas.bind("<Motion>", self.handle_mouse_move)
```

```
Self.canvas.bind("<Button-1>", self.toggle_game_active)
```

```
Self.draw_maze()
```

```
Self.draw_markers()
```

```
Def draw_maze(self):
```

```
For y in range(self.maze.height):
```

```
For x in range(self.maze.width):
```

```
Cell = self.maze.grid[y][x]; x1, y1, x2, y2 = x*self.cell_size,  
y*self.cell_size, (x+1)*self.cell_size, (y+1)*self.cell_size
```

```
If cell['N']: self.canvas.create_line(x1, y1, x2, y1, fill='black', width=  
2)
```

```
If cell['S']: self.canvas.create_line(x1, y2, x2, y2, fill='black', width=  
2)
```

```
If cell['E']: self.canvas.create_line(x2, y1, x2, y2, fill='black', width=  
2)
```

```
If cell['W']: self.canvas.create_line(x1, y1, x1, y2, fill='black', width=  
2)
```

```

Def draw_markers(self):
    Self.goal_pos = (self.maze.width-1, self.maze.height-1)
    Gx, gy = self.goal_pos; cs = self.cell_size
    Self.canvas.create_rectangle(gx*cs+3, gy*cs+3, (gx+1)*cs-3, (gy+
1)*cs-3, fill='green', outline='')

    Px, py = self.player_pos; pad = cs * 0.2
    Self.player_icon = self.canvas.create_oval(px*cs+pad, py*cs+pad, (px+
1)*cs-pad, (py+1)*cs-pad, fill='red', outline='')

    Nx, ny = self.minotaur_pos
    Self.minotaur_icon = self.canvas.create_oval(nx*cs+pad, ny*cs+pad,
(nx+1)*cs-pad, (ny+1)*cs-pad, fill='blue', outline='black', width=2)

Def toggle_game_active(self, event):
    If self.game_over:
        Return
    Self.game_active = not self.game_active
    If self.game_active:
        Self.window.title("Theseus Game – (Active) ESCAPE!")
        # Start both game loops
        Self.move_minotaur()
        Self.update_player_position()
    Else:
        Self.window.title("Theseus Game – Click to Resume")

Def handle_mouse_move(self, event):
    Self.mouse_pos = (event.x, event.y)

Def update_player_position(self):
    If not self.game_active or self.game_over:

```

Return

Cx, cy = self.player_pos

Step 1: Find all valid, open paths from the player's current cell.

Valid_moves = []

Current_cell = self.maze.grid[cy][cx]

If not current_cell['N']: valid_moves.append('N')

If not current_cell['S']: valid_moves.append('S')

If not current_cell['E']: valid_moves.append('E')

If not current_cell['W']: valid_moves.append('W')

Step 2: Determine the direction the mouse is pointing.

Player_center_x = (cx + 0.5) * self.cell_size

Player_center_y = (cy + 0.5) * self.cell_size

Mouse_x, mouse_y = self.mouse_pos

Delta_x = mouse_x - player_center_x

Delta_y = mouse_y - player_center_y

Mouse_direction = None

If abs(delta_x) > abs(delta_y):

 If delta_x > 0: mouse_direction = 'E'

 Else: mouse_direction = 'W'

Else:

 If delta_y > 0: mouse_direction = 'S'

 Else: mouse_direction = 'N'

Step 3: Only move if the mouse direction is one of the valid moves.

If mouse_direction in valid_moves:

```

Nx, ny = cx, cy
If mouse_direction == 'N': ny -= 1
Elif mouse_direction == 'S': ny += 1
Elif mouse_direction == 'E': nx += 1
Elif mouse_direction == 'W': nx -= 1

# Update the player's position
Self.player_pos = (nx, ny)
Pad = self.cell_size * 0.2
Self.canvas.coords(self.player_icon, nx*self.cell_size+pad,
ny*self.cell_size+pad, (nx+1)*self.cell_size-pad, (ny+1)*self.cell_size-pad)
Self.check_game_over()

# Schedule the next player update
Self.window.after(self.player_speed, self.update_player_position)

Def move_minotaur(self):
    If not self.game_active or self.game_over:
        Return

    X, y = self.minotaur_pos
    Cell = self.maze.grid[y][x]

    # 1. Find all physically possible moves (not through a wall)
    Physically_possible = []
    For direction, (dx, dy) in {'N':0,-1, 'S':0,1, 'W':-1,0, 'E':1,0}.items():
        If not cell[direction]:
            Physically_possible.append((x + dx, y + dy))

    # 2. Filter for "forward" moves: not a dead end and not in the current
    path

```



```

Forward_moves = []
For move in physically_possible:
    If move not in self.dead_ends and move not in self.minotaur_path:
        Forward_moves.append(move)

# 3. Decide action based on whether there are forward moves
If forward_moves:
    # We have an unexplored path, so we take it.
    Self.minotaur_pos = random.choice(forward_moves)
    Self.minotaur_path.append(self.minotaur_pos)
Else:
    # No forward moves. We are at a dead end or have looped to our own
path.
    # Mark this location as a dead end and backtrack.
    Self.dead_ends.add(self.minotaur_pos)
    If len(self.minotaur_path) > 1:
        Self.minotaur_path.pop() # Remove current location from path
        Self.minotaur_pos = self.minotaur_path[-1] # Move to the new last
element of the path

# Update minotaur icon on canvas
Nx, ny = self.minotaur_pos
Pad = self.cell_size * 0.2
Self.canvas.coords(self.minotaur_icon, nx*self.cell_size+pad,
ny*self.cell_size+pad, (nx+1)*self.cell_size-pad, (ny+1)*self.cell_size-pad)

Self.check_game_over()
# Schedule the next move
Self.window.after(self.minotaur_speed, self.move_minotaur)

Def check_game_over(self):
    If self.player_pos == self.goal_pos:

```

```

        Self.game_over = True
        MessageBox.showinfo("You Won!", "You escaped the maze!")
        Self.window.destroy()
    elif self.player_pos == self.minotaur_pos:
        Self.game_over = True
        MessageBox.showerror("You Were Caught!", "The minotaur caught you. Game over.")
        Self.window.destroy()

    def run(self):
        Self.window.mainloop()

    def main():
        Parser = argparse.ArgumentParser(description="Generate and play the Theseus maze game.")
        Parser.add_argument('-s', type=int, default=40, help='The size of the maze (e.g., 20 for a 20x20 maze).')
        Parser.add_argument('-d', type=float, default=0.1, help='From 0.0 (hardest) to 1.0 (easiest). Default: 0.1')
        Args = parser.parse_args()
        if args.s < 5:
            Print("Error: Maze size must be at least 5.")
            Return

        Print(f"Generating a {args.s}x{args.s} maze with difficulty {args.d}...")
        Maze = Maze(args.s, args.s, args.d)
        Cell_size = max(5, min(30, 800 // args.s))

        Minotaur_speed_ms = 150 - (args.s * 2)
        Minotaur_speed_ms = max(50, minotaur_speed_ms)

        Game = MazeGame(maze, cell_size, minotaur_speed_ms)

```

```
Game.run()
```

```
If __name__ == "__main__":
```

```
    Main()
```

By default, you get a 40x40 maze, with a few walls broken through to make it easier. You can call it with a -s argument followed by a size (such as 50) to get a bigger maze. A -d argument controls the difficulty. Zero is the most difficult, and 1 is the easiest. The default is 0.1.

Here is what it looks like when the game begins:

Press enter or click to view image in full size

The game as it begins.

The game as it begins.

The red dot is Theseus. The green square is the exit. The blue dot is the Minotaur. You click the mouse (or tap the accelerometer) to begin. Another tap pauses the game to let you figure out the path you want to take.

I left some of the playability a little rough so that it wouldn't be too easy. Feel free to smooth things out some more if you like.

The Mouse

I also had time to write the code for the ESP32-C3 and the MPU6050 (it was a long day waiting for Amazon Next Day delivery).

It would need the `hid_services.py` module and a class to manage the MPU6050 accelerometer. The ESP32-C3 Micropython firmware chose not to include a second I2C driver (presumably to save space?), and the Bluetooth driver uses the first I2C driver, so we use the `SoftI2C` to bit-bang the data out.

The gyros will drift if we don't calibrate them. Keep the device still (sit it on the desk) for a bit before using it. The calibration happens right away in `main()` and lasts a second.

To click the mouse, you tap it (there is only one 'button'). The non-tap movements get smoothed before being reported.

```
Import machine
```

```
Import time
```

```
From machine import SoftI2C
```

```
From hid_services import Mouse
```

```
Class MPU6050:
```

```
    Def __init__(self, i2c):
```

```
        Self.i2c = i2c
```

```
        Self.addr = 0x68
```

```
        # Wake up the MPU6050 from sleep mode
```

```
        Self.i2c.writeto_mem(self.addr, 0x6B, b'\x00')
```

```
        Time.sleep_ms(100)
```

```
    Def _read_word_2c(self, reg):
```

```
        # Reads a 16-bit signed value from two adjacent 8-bit registers
```

```
        High = self.i2c.readfrom_mem(self.addr, reg, 1)[0]
```

```
        Low = self.i2c.readfrom_mem(self.addr, reg + 1, 1)[0]
```

```
        Val = (high << 8) | low
```

```
        Return -((65535 - val) + 1) if val >= 0x8000 else val
```

```
    Def get_gyro_data(self):
```

```
        # Reads gyroscope data (for X/Y movement)
```

```
        Return self._read_word_2c(0x43), self._read_word_2c(0x45)
```

```
    Def get_accel_data(self):
```

```
        # Reads accelerometer data (for tap-to-click)
```

```
        Return self._read_word_2c(0x3B), self._read_word_2c(0x3D), self.
```

```
_read_word_2c(0x3F)
```

```
Def calibrate_gyro(mpu, samples=200):
```

```
    """
```

```
    Reads the gyroscope for a number of samples to determine the offset.
```

```
    This should be done with the sensor held perfectly still.
```

```
    """
```

```
    Print("Calibrating gyroscope... Keep the device perfectly still!")
```

```
    Offset_x = 0
```

```
    Offset_y = 0
```

```
    For _ in range(samples):
```

```
        Gx, gy = mpu.get_gyro_data()
```

```
        Offset_x += gx
```

```
        Offset_y += gy
```

```
        Time.sleep_ms(5)
```

```
    Offset_x /= samples
```

```
    Offset_y /= samples
```

```
    Print(f"Calibration complete. Offset X: {offset_x:.2f}, Offset Y: {offset_y:.2f}")
```

```
    Return offset_x, offset_y
```

```
Def main():
```

```
    Print("Initializing I2C and MPU6050 sensor...")
```

```
    I2c = SoftI2C(scl=machine.Pin(7), sda=machine.Pin(6), freq=400000)
```

```
    Mpu = MPU6050(i2c)
```

```
    Print("Sensor initialized.")
```

```
    Gyro_offset_x, gyro_offset_y = calibrate_gyro(mpu)
```

```

Print("Initializing BLE HID Mouse Service...")
Mouse = Mouse("ESP32-C3 Mouse")
Print("Mouse service created.")

Print("Starting BLE services...")
Mouse.start()
Print("Manually starting advertising...")
Mouse.start_advertising()
Print("Device is now advertising as 'ESP32-C3 Mouse'. Ready to connect.")

SENSITIVITY = 400    # Lower value = more sensitive
DEADZONE = 50        # Ignores small movements to prevent drift
TAP_THRESHOLD = 30000 # Acceleration needed for a click. Tune this!

ALPHA = 0.3          # Smoothing factor (0.0 to 1.0). Smaller is smoother.
Smoothed_dx = 0.0     # Initialize smoothed values as floating point
numbers
Smoothed_dy = 0.0

Last_tap_time = 0

Try:
    While True:
        If mouse.is_connected():
            B1, b2, b3 = False, False, False
            Dx, dy = 0, 0

            If time.ticks_diff(time.ticks_ms(), last_tap_time) > 250:
                Ax, ay, az = mpu.get_accel_data()

```

```

        If (ax*ax + ay*ay + az*az) > (TAP_THRESHOLD *
TAP_THRESHOLD):
            Print("Tap Detected! Ignoring simultaneous movement.")
            B1 = True
            Last_tap_time = time.ticks_ms()
        Else:
            # --- THIS IS A MOVEMENT EVENT ---
            Gyro_x, gyro_y = mpu.get_gyro_data()

            Calibrated_gx = gyro_x - gyro_offset_x
            Calibrated_gy = gyro_y - gyro_offset_y

            Raw_dx, raw_dy = calibrated_gy, -calibrated_gx

            Smoothed_dx = (raw_dx * ALPHA) + (smoothed_dx * (1 -
ALPHA))

            Smoothed_dy = (raw_dy * ALPHA) + (smoothed_dy * (1 -
ALPHA))

            Dx = int(max(min(smoothed_dx / SENSITIVITY, 127), -127)) if
abs(smoothed_dx) > DEADZONE else 0
            Dy = int(max(min(smoothed_dy / SENSITIVITY, 127), -127)) if
abs(smoothed_dy) > DEADZONE else 0

            Mouse.set_axes(dx, dy)
            Mouse.set_buttons(b1, b2, b3)

            Mouse.notify_hid_report()

        If b1:
            Mouse.set_buttons(False, False, False)
            Mouse.notify_hid_report()

```

```
Time.sleep_ms(20)
```

Except KeyboardInterrupt:

```
Print("Caught keyboard interrupt. Stopping services.")
```

Finally:

```
Mouse.stop()
```

```
Print("Services stopped.")
```

Main()

I quickly realized this was not going to be a teaching vehicle for a schoolboy. But the general concepts, that there can be a program running on his laptop that he could control by radio from a computer in his hand, that part he could grasp.

And that a programmer could build it in a (long) day while waiting for parts.

130